SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence	**COURSE CODE: CSA1709**
COURSE OUTCOME COVERED CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3) Assessment Tool Weightage: 35%	

QUESTIONS: 5 **Duration: 1 Hour**

Total Marks: 50

Annexure A

Analytical Problem Solving

Code of Conduct:

I V. Vishwa Tej certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V. Vishwa Tej

1. Water Jug Problem

Problem statement. Two unmarked jugs have capacities of 4 gallons and 3 gallons. A pump can fill a jug, water can be discarded, and water can be transferred between jugs. The goal is to obtain exactly 2 gallons in the 4-gallon jug without any additional measuring device.

State representation and operators

Represent each state as (x, y), where x is the amount in the 4-gallon jug and y is the amount in the 3-gallon jug. The initial state is (0, 0), and the goal test is x = 2. Legal operators are filling either jug, emptying either jug, and pouring water from one jug to the other until the source is empty or the receiving jug is full.

Step	Action	4-gallon jug	3-gallon jug	State
0	Start with both empty	0 gal	0 gal	(0, 0)
1	Fill the 3-gallon jug	0 gal	3 gal	(0, 3)
2	Pour 3-gallon jug into 4-gallon jug	3 gal	0 gal	(3, 0)
3	Fill the 3-gallon jug again	3 gal	3 gal	(3, 3)
4	Pour into 4-gallon jug until full (only 1 gal fits)	4 gal	2 gal	(4, 2)
5	Empty the 4-gallon jug	0 gal	2 gal	(0, 2)
6	Pour remaining 2 gal into the 4-gallon jug	2 gal	0 gal	(2, 0)

Solution path: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0)

Conclusion: Exactly 2 gallons are obtained in the 4-gallon jug using only the allowed operations.

2. Mars Rover as an Intelligent Agent

A Mars rover is an autonomous or semi-autonomous agent that senses its surroundings, chooses actions, and performs scientific exploration. It operates with incomplete information and must account for terrain, limited energy, communication delays, and equipment safety.

i. Percepts

Percept source	Information received	Use
Cameras	Images of rocks, terrain, slopes, obstacles and sample locations	Navigation and visual inspection
Scientific instruments	Rock/soil composition, chemical signals and sample measurements	Select scientifically useful targets
Motion and position sensors	Wheel movement, orientation, position estimates and tilt	Localisation and safe driving
Health and environment sensors	Battery level, temperature, equipment status and environmental conditions	Protect rover and manage resources

ii. Operating environment

Property	Characterisation
Observability	Partially observable: sensors cannot reveal every hidden property of terrain or samples.
Determinism	Uncertain: wheel slip, dust, terrain and instrument readings can vary.
Dynamics	The environment changes slowly in many respects, but movement and hazards require timely decisions.
State/action space	Continuous position and movement, with discrete choices such as target selection and sample analysis.

Other agents	Primarily a single-agent task, with human mission

iii. Actions available

Move forward or turn; stop and avoid an obstacle; capture images; select a rock or soil target; position an instrument; collect or handle a sample when equipped; analyse a sample; transmit findings; request instructions; and enter a safe mode when power or equipment conditions require it.

iv. Performance evaluation

Evaluate scientific value and reliability of data, number and quality of samples analysed, progress toward mission targets, navigation safety, energy efficiency, successful communication, equipment health, and completion of assigned objectives. The rover should maximise useful science while minimising risk and unnecessary resource use.

2. Mars Rover — Suitable Agent Architecture

v. Recommended architecture: model-based, goal-based agent with utility-based decision-making

A purely reflex agent reacts only to current percepts and may make poor decisions when hazards are partly hidden or actions have delayed consequences. A model-based agent maintains an internal state of the rover, terrain, known obstacles, targets, resources and uncertainty. A goal-based layer plans toward mission goals, while a utility measure helps choose between competing safe actions.

Decision cycle

Stage	Rover activity
1. Sense	Collect camera, position, scientific and health data.
2. Update model	Update estimated location, map, target properties, battery and uncertainty.
3. Set/confirm goal	Choose the next science target or safety objective based on mission priorities.
4. Plan and compare	Estimate action outcomes, travel cost, energy use, scientific value and risk.

5. Act and monitor	Execute a safe action, check its outcome, and re-plan when new percepts arrive.

Why this architecture fits

- Memory preserves information that cannot be observed in a single image or sensor reading.
- Planning selects sequences of actions rather than only immediate reactions.
- Adaptation revises the plan when terrain, sample results or resources differ from expectations.
- Safety and utility balance scientific reward against energy, time and damage risk.

Illustrative agent loop

```
WHILE mission is active:
  percepts ← read_sensors()
  internal_state ← update_model(internal_state, percepts)
  goal ← select_next_goal(internal_state, mission_plan)
  plan ← choose_plan(internal_state, goal, energy, risk)
  action ← first_action(plan)
  execute(action)
  IF hazard_detected OR plan_failed: replan_or_enter_safe_mode()
```

3. Eight-Queens Problem: Search-Space Formulation

The task is to place eight queens on an 8×8 chessboard so that no two queens attack each other. A queen attacks along its row, column and diagonals.

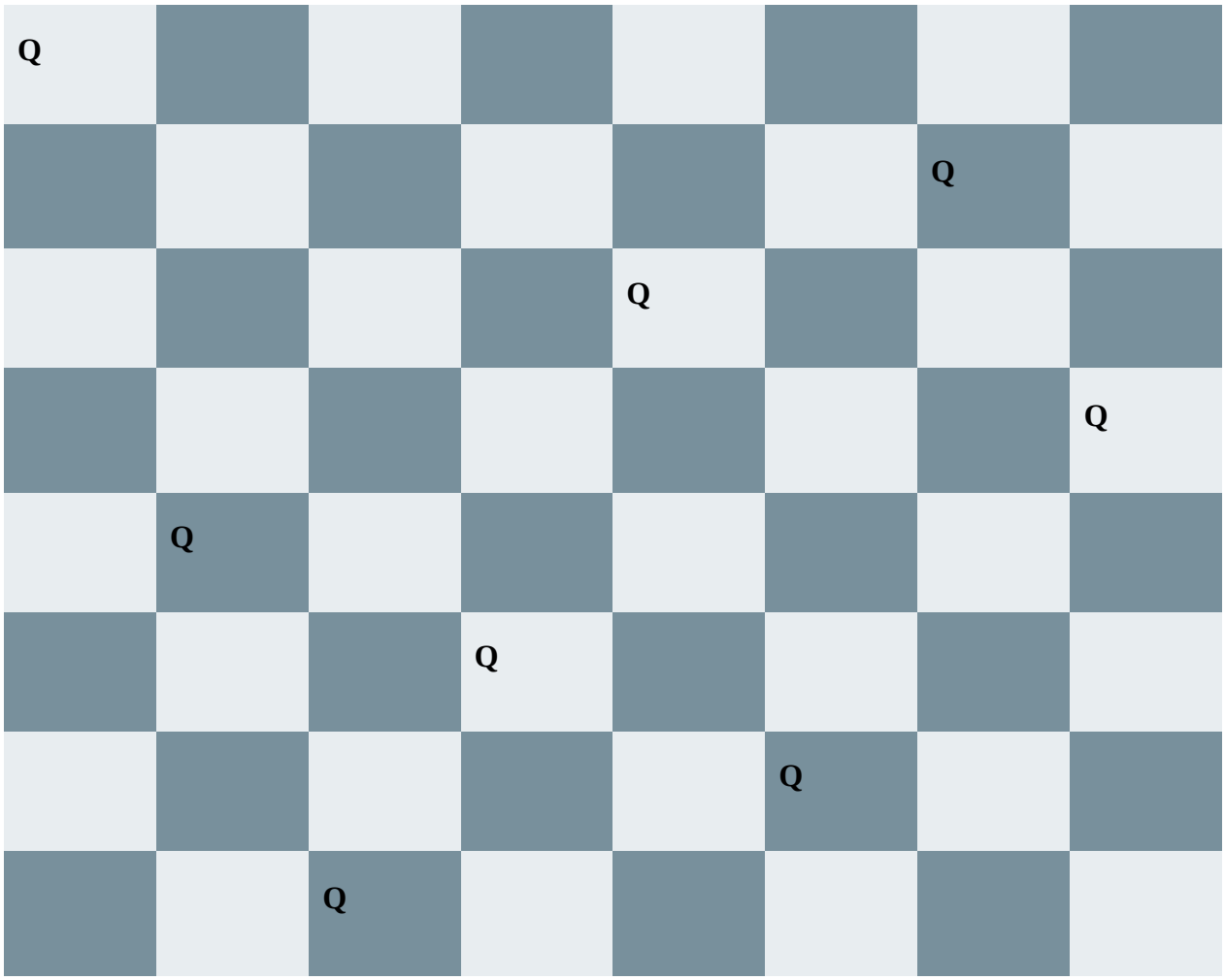
Problem components

Component	Formulation
Initial state	An empty chessboard, with no queens placed.
State	A partial assignment of queens to columns; the index is the column and the value is the row.
Actions	Place a queen in a legal square of the next column.
Transition model	Add one queen to the selected square while preserving previous placements.
Goal test	Eight queens are placed and no pair shares a row, column or diagonal.
Path cost	Each placement may have unit cost; the objective is to find any valid goal state.

Useful constraints

If one queen is placed in each column, column conflicts are prevented by construction. For queens at (r_1,c_1) and (r_2,c_2), require $r_1 \neq r_2$ and $|r_1 - r_2| \neq |c_1 - c_2|$. These conditions prevent same-row and diagonal attacks.

Illustrative valid arrangement



Search strategy and pseudocode

Backtracking depth-first search places a queen in the next column, checks conflicts, and continues only if safe. If no row is safe, it removes the previous queen and tries another row.

```
SOLVE(column):
  IF column = 9: return success
  FOR each row from 1 to 8:
    IF square(row, column) is safe:
      place queen(row, column)
      IF SOLVE(column + 1): return success
      remove queen(row, column)
  return failure
```

4. OLA Cab Booking as a Problem-Solving Agent

The customer enters a pickup location and destination and may select a cab category such as Mini, Micro, Sedan, Shared or Prime. A goal-based problem-solving agent can search for a feasible booking plan, with utility-based selection when several options are available.

Problem formulation

Element	OLA booking interpretation
Initial state	Customer has opened the app; pickup and destination may not yet be confirmed.
Goal state	A suitable cab is selected and booking is confirmed, or the user is informed that no option is available.
Actions	Enter locations, validate them, request cab categories, compare fare/ETA, select, confirm, edit or cancel.
Constraints	Locations must be valid; cab must be available; choice should respect preferences and displayed fare.
Evaluation	Successful booking, suitable ETA and fare, preference match, and clear handling of unavailable options.

Pseudocode

```
BEGIN
  pickup ← get_pickup_location()
  destination ← get_destination()
  IF pickup is invalid OR destination is invalid:
    display("Please enter valid locations")
    STOP
  preferences ← get_user_preferences()
  cab_options ← request_available_cabs(pickup, destination)
  IF cab_options is empty:
    display("No cabs available; try again later")
    STOP
  feasible_options ← filter(cab_options, preferences)
  IF feasible_options is empty:
    feasible_options ← cab_options
```

```
  display_fare_and_ETA(feasible_options)
  selected_cab ← get_customer_selection(feasible_options)
  IF selected_cab is confirmed by customer:
    booking ← confirm_booking(selected_cab, pickup, destination)
    display_booking_status(booking)
  ELSE:
    display("Booking not confirmed")
END
```

Decision flow

Input	Decision / action	Outcome
Locations	Validate pickup and destination	Continue only when valid
Cab request	Fetch available categories, fares and ETAs	Show choices or report none
Preferences	Filter by category, budget or comfort	Present suitable options
Confirmation	Ask customer to confirm choice	Create booking and show status

This is a conceptual assessment model. Real booking requires live location, availability, fare and payment services.

5. Uniform Cost Search (UCS) for the Delivery Network

The logistics company wants the least-cost route from start node S to goal node G. Edge weights represent travel cost combining fuel and time. UCS expands the frontier node with the lowest accumulated path cost g(n), not the node with the fewest edges.

Weighted graph from the question

Edge list for the supplied network: S→A (1), S→G (12), A→B (3), A→C (1), B→D (3), C→D (1), C→G (2), D→G (3).

Edge	Cost	Edge	Cost
S → A	1	S → G	12
A → B	3	A → C	1
B → D	3	C → D	1
C → G	2	D → G	3

UCS procedure

1. Put S in a priority queue with cost 0.
2. Remove the node with the smallest accumulated cost.
3. If it is G, return the path and cost.
4. Otherwise expand outgoing edges and insert/update successors with cumulative cost.
5. Ignore a more expensive route to a state when a cheaper route is already recorded. Repeat until G is removed or the queue is empty.

Expansion trace

Step	Expanded node	Candidate paths / costs	Frontier (lowest first)
1	S (0)	A = 1; G = 12	A(1), G(12)
2	A (1)	B = 1+3 = 4; C = 1+1 = 2	C(2), B(4), G(12)
3	C (2)	D = 2+1 = 3; G = 2+2 = 4	D(3), B(4), G(4), G(12)
4	D (3)	G = 3+3 = 6; route via B costs more	B(4), G(4), G(6), G(12)
5	B (4)	D via B = 4+3 = 7, worse than known cost 3	G(4), G(6), G(12)
6	G (4)	Goal reached; stop	—

Least-cost result: S → A → C → G; total cost = 1 + 1 + 2 = 4.

Candidate route comparison

Route	Calculation	Total
S → A → C → G	1 + 1 + 2	4
S → A → C → D → G	1 + 1 + 1 + 3	6
S → A → B → D → G	1 + 3 + 3 + 3	10
S → G	12	12

UCS is appropriate because the edge costs are non-negative and the goal is to minimise cumulative cost. The optimal path in the supplied network has cost 4.

Summary and Assessment Checklist

Summary of answers

Question	Main concept	Key result
1. Water Jug	State-space search and legal operators	Reach state (2, 0): exactly 2 gallons in the 4-gallon jug.
2. Mars Rover	Intelligent-agent design	Use percepts, an environment model, goals and utility to explore safely.
3. Eight Queens	Constraint satisfaction and search	Backtracking with row and diagonal checks finds a valid board.
4. OLA Booking	Goal-based problem-solving agent	Validate locations, compare cab choices and confirm a booking.
5. Logistics UCS	Lowest cumulative-cost search	Optimal route S → A → C → G with cost 4.

Quality checks before submission

- Each solution identifies the task and method used.
- State transitions, constraints and path costs are shown explicitly where relevant.
- Diagrams and tables support the written explanation.
- The final result is stated clearly for every question.
- Complete student name, registration number and signature on the cover page.

Rubric supplied with the question paper

Criterion	Weightage	Strong response demonstrates
Problem understanding	20%	Identifies all parameters and requirements accurately.
Method selection	20%	Chooses the appropriate method and justifies it.
Solution process	25%	Uses logical, systematic steps throughout.
Accuracy of calculation	20%	Shows accurate calculations.